CatSem

Certified optimization from categorical semantics

Project overview

12 November 2026

CatSem explores how categorical logic can guide an executable, verified optimizer. The core question is practical: **what extra condition must hold before a rewrite is safe, and how can that condition be moved across assignments, branches, and loops?** The project develops the abstract answer in Lean 4, implements compiler passes over small intermediate languages, and proves that those passes preserve an independent concrete evaluator.

This is a research compiler core: its current executable scope covers integer expressions, structured imperative blocks, and fuel-indexed loops. It is not yet a compiler from a source language to verified machine code.

Snapshot: 12 November 2026. The integrated checkout is at `b8e8eca` before this documentation change. Milestones 1–3 have recorded approved implementation audits. Milestone 4 is an active specification draft in the theory worktree. The PDF companion and editable LaTeX source are in `docs/catsem-overview.pdf` and `docs/catsem-overview.tex`.

The motivating example

Consider `(x * d) / d`. Replacing it by `x` is valid when `d != 0`, but erases a checked division trap when `d = 0`. A correct optimizer needs both an algebraic identity and evidence that its side condition holds.

That evidence can come from an entry assumption, a branch condition, or a previous successful checked division. In `(x / d) + ((y * d) / d)`, reaching the right operand after a successful left division establishes that the same divisor is nonzero. The optimizer can reuse that fact. If the left division traps, strict evaluation must still stop there.

Across `d := x`, the requirement that the new `d` be nonzero becomes a requirement on the old `x`. Across a loop, checking only the initial divisor is insufficient if the latch can later make it zero. These three situations motivate guard synthesis, predicate pullback, and inductive invariant strengthening.

The mathematical spine

Predicates form a Heyting algebra. Think of an element as a condition on possible machine states. The order `a <= b` means that `a` implies `b`; meet means conjunction. Given known facts `a` and a desired safety property `b`, the synthesized guard is Heyting implication, `a => b`. Its universal property is:

```
a AND g <= b iff g <= (a => b)
```

Thus the synthesized guard is the weakest sufficient extra condition: every other sufficient guard implies it. This is a semantic characterization, not a claim that the generated syntax has the smallest runtime cost.

Program transitions transport predicates. `CFGTransfer` pairs a forward strongest-postcondition map with a backward weakest-precondition map. Their Galois connection says:

```
post(a) <= b iff a <= pullback(b)
```

For a concrete deterministic step, pullback evaluates the target predicate after that step; the forward map existentially describes reachable output states. Sequential pullbacks compose in reverse execution order. To establish `b` after a transition while assuming `a` before it, synthesize `a => pullback(b)`.

Equivalence is conditional. `Forces` is defined as entailment at a stage, and `GuardedEquiv` requires matching evaluations at stages that establish a guard. The earlier proof rounds develop guarded rewrite chains, congruence, branch/diamond transport, and loop transport. The concrete `stateTransfer` and `liftEval` constructions connect this layer to state predicates and operational evaluation. The code uses Kripke–Joyal terminology; readers should follow these explicit definitions rather than assume a complete general-purpose topos or sheaf library.

Loops need inductive evidence. An invariant must imply the desired safety property and survive a loop step. Once such a predicate is available, its pullback through initialization yields the preheader requirement. A runtime versioning guard can choose an optimized loop when that requirement holds and a conservative version otherwise.

From mathematical specification to executable passes

The abstract logic lives alongside concrete syntax and evaluators in `Specs/Defs.lean`. `GuardIR` is finite executable syntax with a Boolean evaluator and a denotation into predicates on environments. `PureExpr` supplies total arithmetic suitable for guards; `Expr` adds checked division, unchecked division, and guarded branches.

`Outcome` distinguishes values, checked traps, and unchecked undefined behavior. Correctness requires exact outcome equality under the supplied entry context. In particular, a trap cannot silently become unchecked UB, and multiplying an expression by zero cannot erase an earlier failure. Arithmetic uses unbounded Lean integers and Euclidean division, not fixed-width arithmetic or C-style signed division.

The implemented compiler functions are total computable Lean definitions. The charter permits only `propext` and `Quot.sound` as kernel axioms, excluding `sorryAx` and `Classical.choice` from accepted implementations and proofs. Target stubs are a specification interface; they are not evidence of completed proofs. The accepted implementation and its dependencies must pass the separate verification checks.

Work completed so far

Milestone 1: guarded expressions

`synthesizeGuardIR` computes residual guards, while `optimizeExpr` performs guarded cancellation, pure arithmetic fusion, and safe elimination of repeated checks. Context facts and facts established by successful earlier evaluation guide recursive rewriting. The semantic bridge connects these operational transformations to the abstract guarded-equivalence development.

The M1 audit records 23 named test theorems and a bounded sweep of 768 conditional cases. Its independent compound example preserves value 15 while reducing two checked divisions to one under an unknown context, or to zero with sufficient nonzero assumptions. Negative cases cover distinct divisors, zero inner quotients, strict failures, and facts that must not escape an unexecuted branch.

Milestone 2: assignments and diamonds

`pullbackGuardIR` computes preconditions through statements. `synthesizeStmtGuardIR` synthesizes residual entry guards, and `optimizeBlock` optimizes statement prefixes and terminal expressions. Assignment substitution, branch joins, and changing conditions matter: a fact about an old variable value must not be reused after that variable is overwritten.

The M2 audit records 20 added test theorems, 624 pullback comparisons, 3,120 synthesis comparisons, and 420 block/context combinations. Representative abstract check costs fall from 2 to 0 for saved-divisor and dead-branch cases, from 3 to 1 at asymmetric joins, and from 4 to 1 in nested branches. Unsafe paths retain their checks.

Milestone 3: invariant strengthening and loop hoisting

`havocGuardIR` conservatively accounts for arbitrary accumulator writes. `strengthenLoopInvariantIR` finds sufficient inductive facts, `hoistLoopGuardIR` transports them through initialization, and `optimizeLoop` builds optimized or versioned loop programs. Body and exit expressions are analyzed under their respective loop conditions.

The M3 audit records 26 added test theorems and 140 comparisons across loop/context/state cases and fuel values. Counted-loop check costs fall from 7 to 3 with a known divisor, or from 7 to 4 with preheader versioning. A five-stage pipeline falls from 11 to 6. Tests also preserve zero-trip behavior, later zero-divisor traps, accumulator clobbering, signed arithmetic, and exact fuel exhaustion.

These numbers are recorded audit examples, not wall-clock speedups or universal cost bounds. The general semantic proofs cover arbitrary inputs satisfying their hypotheses; bounded executable sweeps provide complementary concrete evidence.

How the three-agent workflow fits together

The workspace separates responsibilities across Git worktrees:

- `wt-antigravity` / `agent/theory-spec`: theory, definitions, target statements, proof dependency blueprint, and verification tooling.

- `wt-claude` / `agent/prover`: executable implementation and constructive proofs in `Proofs/Submission.lean`.
- `wt-codex` / `agent/redteam`: adversarial specification tests and independent implementation audits.
- `main` / `master`: integrated results, including the three completed milestone commits.

Gate 1 examines the specification before locking it: do the definitions express the intended problem, are assumptions satisfiable, and can degenerate implementations meet the targets? Gate 2 examines the actual implementation, including independent computations and failure-preservation tests. Positive reduction witnesses prevent an identity optimizer from satisfying the project merely by being correct.

The worktrees contain substantial uncommitted work, so branch commit IDs alone do not describe their current contents. Synchronization and audit hashes matter. At this snapshot the integrated submission digest matches the M3 approval: `ba5aeefca9f932707e8e24ca27029ce2717882a48ab2dc68f1b13103528fc5bf`. Earlier audit digests refer to earlier submissions and should be read historically.

Reading and running the project

Start with `CHARTER.md` for the contract, then `Specs/Defs.lean` for the semantic model. Read `Specs/Target.lean` alongside `Proofs/Submission.lean` to compare required and delivered behavior. `Specs/Verify.lean` checks the submission interface; `Specs/RedTeam.lean` and `Specs/PostImplAudit.lean` hold the two audit suites. `blueprint/lemma_dag.json` records claims and dependencies; its status labels alone are not proof of implementation completion. The narrative audit reports are under `reviews/`.

The checkout pins Lean `v4.35.0` and uses Mathlib through Lake. With the pinned toolchain and dependencies available, run from the repository root:

```
lake build Specs Proofs.Submission

lake env lean Specs/PostImplAudit.lean
```

For the repository's fuller gate workflow:

```
bash scripts/verify.sh -spec-audit

bash scripts/verify.sh -kernel-only

bash scripts/verify.sh
```

The default and `-kernel-only` modes copy the candidate submission from `../wt-claude` and audit files from `../wt-codex` before verification. Default mode also imports the M3 approval report. These commands mutate the integrated checkout and depend on the sibling layout; use them deliberately. `-spec-audit` runs the specification gate without that initial candidate import. The gate tooling checks axiom logs, binds module artifacts to hashes, and replays modules with `leanchecker`.

This documentation pass inspected source, history, reports, and the integrated submission hash; it did not rerun the Lean gates. Recorded approvals are attributed to their reports, whose implementation reviews explicitly exclude auditing the build scripts themselves.

Active next milestone: scope and quantification

Frontier records `m4-spec-draft` as open. The theory worktree contains a draft titled "First-Order Hyperdoctrine, Beck–Chevalley & Scoped letBind Optimizer." This is work in progress, not an accepted M4 implementation.

The draft extends the logic from changing machine states to changing variable scopes. A scope projection forgets a bound variable; reindexing brings a predicate into a larger scope. Existential and universal quantification sit on either side of that reindexing operation as adjoints. Beck–Chevalley laws express that suitable substitutions commute with quantification, making scope changes compositional.

The proposed executable counterpart uses scope-indexed expressions and strict `letBind` evaluation. Its targets include safe substitution and weakening, nested bindings, reuse of facts learned from successful effectful right-hand sides, and preservation of an unused binding whose right-hand side traps. The connection to the earlier work is direct: safety facts must survive both state changes and scope changes without variable capture or lost effects.

The immediate path is to finish and audit the M4 specification, lock the obligations, implement the scoped passes and proofs, and run independent post-implementation tests before integration. The active draft may change during that process.

Longer-term goals and open limits

The following are suggested priorities, distinct from the active M4 draft:

- Improve optimization precision with proved arithmetic normalization and richer branch facts. M2 records symbolic arithmetic that remains unsimplified; correctness currently takes priority over completeness.
- Make invariant discovery more capable while retaining a total algorithm. M3 uses eight conservative saturation rounds and size-limited joins; a nine-stage example demonstrates that safe opportunities can remain undiscovered.
- Measure compilation time, guard size, and end-to-end execution costs. Current observers count specified IR checks, not all runtime work, and do not establish global optimality.
- Broaden the semantic boundary deliberately: fixed-width overflow, alternative signed division, richer control flow, and eventually a backend each require explicit semantics and new proofs. Binary machine-code verification is deferred by the charter.
- Improve reproducibility by making candidate import and verification separate steps, recording accepted snapshots, and keeping specification, implementation, and audit hashes visibly aligned.

The central goal remains the charter's: an executable compiler certified as a constructive realization of categorical structures, paired with operational equivalence proofs over an independent concrete evaluator.